VHDL Combinational Logic Simulation

1. Four-Input AND and OR Gates

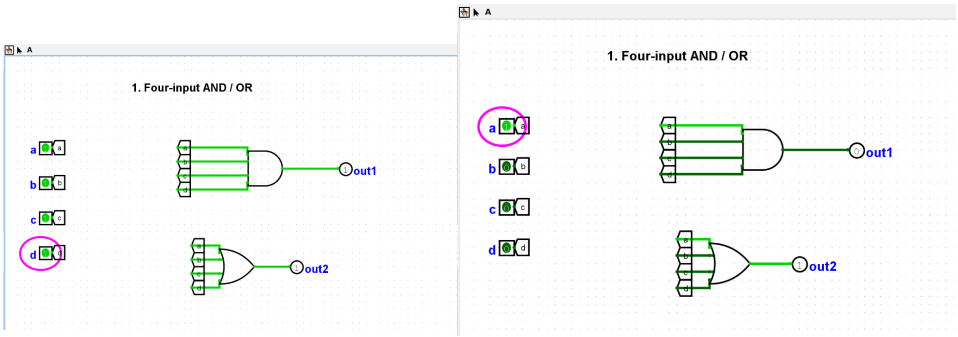
1.1 VHDL Code

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity logic4 is
 port (a,b,c,d : in std_logic; out1,out2 : out std_logic);
end entity;
architecture rtl of logic4 is

begin
 out1 <= a and b and c and d;
 out2 <= a or b or c or d;
end architecture;
```

1.2 Logisim-evolution Simulation



Input : a = 1, b = 1, c = 1, d = 1
Output : out1 = 1, out2 = 1
When all four inputs are 1, both the AND output and the OR output become 1.

Input : a = 1, b = 0, c = 0, d = 0
Output: out1 = 0, out2 = 1
The AND output becomes 0 because not all inputs are 1. The OR output remains 1 because at least one input is 1.

1.3 Simulation Results

a	b	c	d	AND (out1)	OR (out2)
0	0	0	0	0	0
1	0	0	0	0	1
1	1	1	0	0	1
1	1	1	1	1	1

1.4 Conclusion

The outputs match the truth tables of four-input AND and OR gates.

2. 8-to-1 Multiplexer

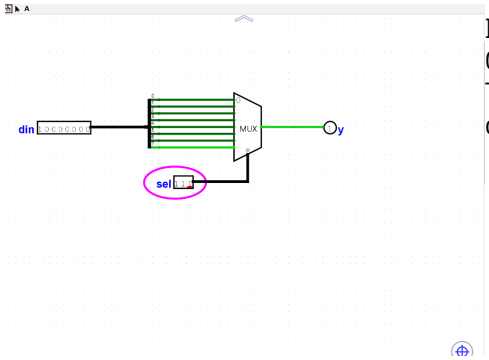
2.1 VHDL Code

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity mux8 is
 port (din : in std_logic_vector(7 downto 0); sel : in std_logic_vector(2 downto 0); y : out std_logic);
end entity;
architecture rtl of mux8 is

begin
 process(all) begin
  case sel is
   when "000" => y <= din(0);
   when "001" => y <= din(1);
   when "010" => y <= din(2);
   when "011" => y <= din(3);
   when "100" => y <= din(4);
   when "101" => y <= din(5);
   when "110" => y <= din(6);
   when "111" => y <= din(7);
   when others => y <= '0';
  end case;
 end process;
end architecture;
```

.2.2 Logisim-evolution Simulation



Input: din = 10000000, sel = 111
Output: y = 1
The selection input 111 selects din(7). Since din(7) is 1, the output becomes 1.

2.3 Simulation Results

For din = 00001000, the output is 1 only when sel = 011. For all other selection values, the output is 0.

2.4 Conclusion

The multiplexer correctly transfers one of eight inputs to the output according to the three-bit selection signal.

3.1 VHDL Code

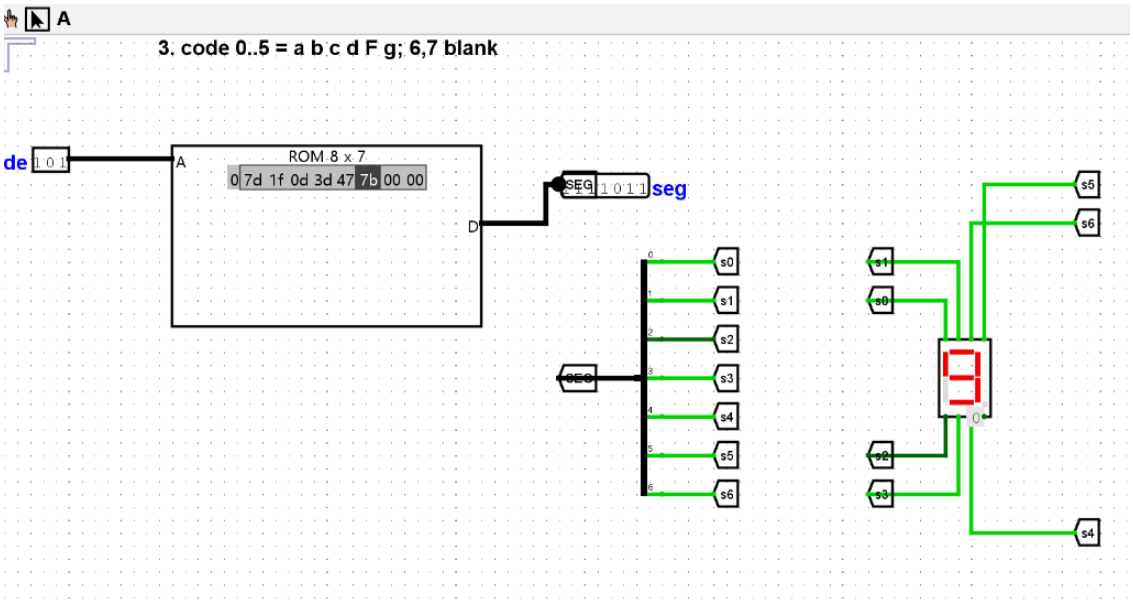
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity letters7 is
 port (code : in std_logic_vector(2 downto 0); seg : out std_logic_vector(6 downto 0));
end entity;
architecture rtl of letters7 is

begin
 -- seg(6 downto 0) = a,b,c,d,e,f,g; active high.
 process(all) begin
  case code is
   when "000" => seg <= "1111101";
   when "001" => seg <= "0011111";
   when "010" => seg <= "0001101";
   when "011" => seg <= "0111101";
   when "100" => seg <= "1000111";
   when "101" => seg <= "1111011";
   when "110" => seg <= "0000000";
```

```
    when "111" => seg <= "0000000";
    when others => seg <= (others => '0');
  end case;
 end process;
end architecture;
```

3.2 Logisim-evolution Simulation



Input: code = 101
Output: Letter g
When code is 101, the display shows the letter g using a pattern similar to the digit 9.

3.3 Simulation Results

Input code	Display
000	a
001	b
010	c
011	d
100	F
101	g
110	Off
111	Off

The letter f is represented as F, and g uses a pattern similar to the digit 9.

3.4 Conclusion

The circuit displays the selected letters using the corresponding seven-segment patterns.

4. 25-Person Voting Circuit

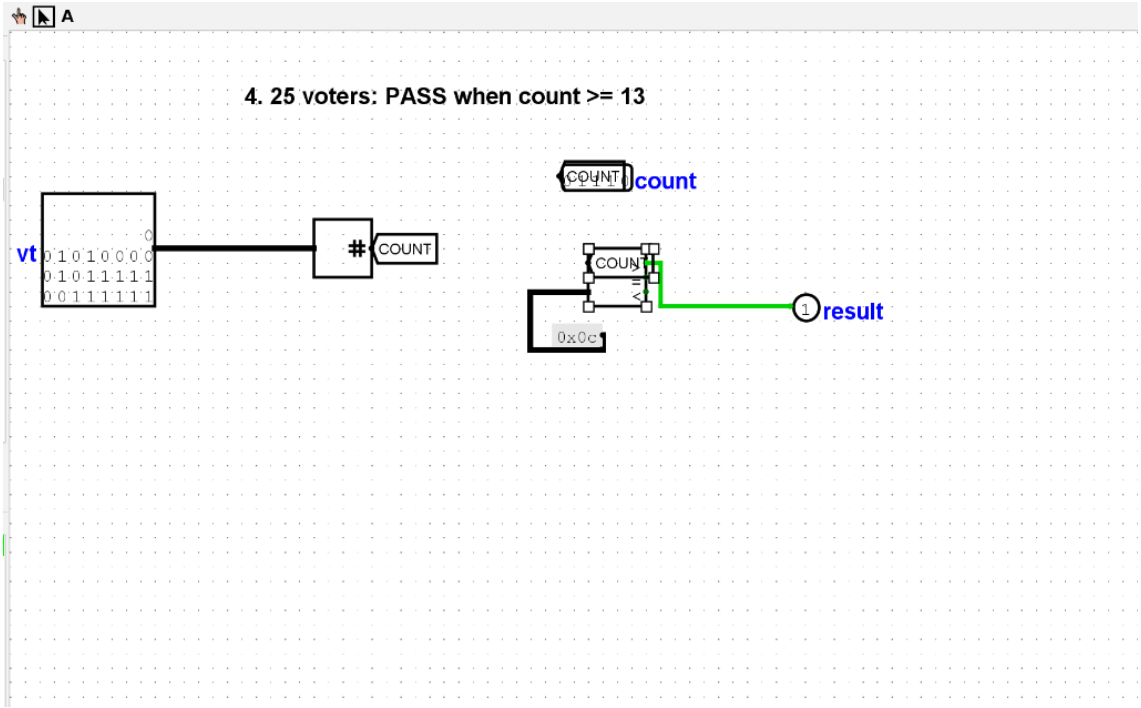
4.1 VHDL Code

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity vote25 is
 port (vt : in std_logic_vector(24 downto 0); count : out std_logic_vector(4 downto 0); result : out std_logic);
end entity;
architecture rtl of vote25 is

begin
 process(all)
  variable n : natural range 0 to 25;
 begin
  n := 0;
  for i in vt'range loop
   if vt(i) = '1' then n := n+1; end if;
  end loop;
  count <= std_logic_vector(to_unsigned(n,5));
  if n >= 13 then result <= '1'; else result <= '0'; end if;
 end process;
end architecture;
```

4.2 Logisim-evolution Simulation



Input: 12 affirmative votes
Output: count = 01100, result = 0
When 12 people vote in favor, the proposal does not receive a majority.

Input: 13 affirmative votes
Output: count = 01101, result = 1
When 13 people vote in favor, the proposal receives a majority.

4.3 Simulation Results

Affirmative votes	Result
0-12	0: Rejected
13-25	1: Accepted

Each input bit represents one vote: 1 means in favor, and 0 means against. A simple majority threshold of 13 votes is used.

4.4 Conclusion

The circuit correctly counts the affirmative votes and accepts the proposal when at least 13 of the 25 votes are in favor.

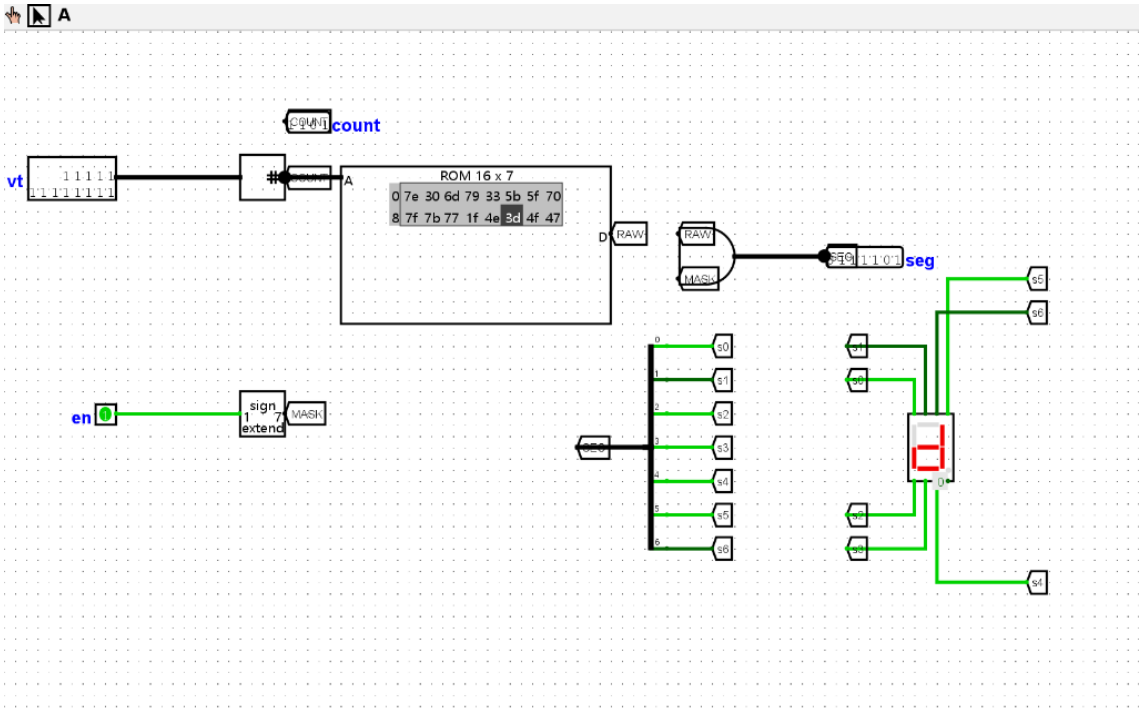
5. 13-Bit Population Count and Seven-Segment Display

5.1 VHDL Code

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity popcount13_display is
 port (vt : in std_logic_vector(12 downto 0); en : in std_logic; count : out
std_logic_vector(3 downto 0); seg : out std_logic_vector(6 downto 0));
end entity;
architecture rtl of popcount13_display is
 signal cnt : std_logic_vector(3 downto 0);
 signal decoded : std_logic_vector(6 downto 0);
begin
 process(all)
  variable n : natural range 0 to 13;
 begin
  n := 0;
  for i in vt'range loop
   if vt(i) = '1' then n := n+1; end if;
  end loop;
  cnt <= std_logic_vector(to_unsigned(n,4));
 end process;
 count <= cnt;
 decoder: entity work.hex7 port map(value => cnt, seg => decoded);
 seg <= decoded when en = '1' else (others => '0');
end architecture;
```

5.2 Logisim-evolution Simulation



Input: vt = 1111111111111, en = 1
Output: count = 1101, display = d
All thirteen input bits are 1. The circuit counts thirteen 1s and displays d, which represents the hexadecimal value D.

5.3 Simulation Results

Number of 1s	Enable	Display
0–9	1	0–9
10	1	A
11	1	b
12	1	C
13	1	d
Any value	0	Off

A 13-bit input can contain at most thirteen 1s. Therefore, E and F cannot occur in the integrated circuit, although the VHDL hexadecimal decoder supports them.

5.4 Conclusion

The circuit correctly counts the 1s in the 13-bit input and displays the corresponding digit or letter. The enable signal controls whether the display is on or off.

The VHDL implementation consists of two files: popcount13_display.vhd and hex7.vhd. The first module counts the number of 1s and controls the display enable signal. The hex7 module converts the count into the corresponding seven-segment pattern. Both source files are included in the GitHub repository.